

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №4 (Яндекс.Контест)

Выполнил:

Студент группы Р3210

Бармичев Григорий Андреевич

Преподаватель:

Косяков Михаил Сергеевич

Санкт-Петербург, 2026г.

Задача №13 «М. Цивилизация»

Описание:

Дана карта размера $N \times M$, состоящая из полей, лесов и воды. Нужно провести поселенца из начальной клетки в конечную так, чтобы суммарное время пути было минимальным: переход в поле стоит 1, в лес 2, в воду переходить нельзя. Требуется вывести минимальное время и любой подходящий маршрут.

Доказательство:

Рассмотрим клетки карты как вершины графа, а переходы в соседние по стороне клетки, как рёбра. Вес ребра равен стоимости перехода в соответствующую клетку: 1 для поля и 2 для леса. Так как все веса неотрицательны, для поиска кратчайшего пути подходит алгоритм Дейкстры.

В массиве `state` для каждой клетки хранится минимальная стоимость достижения, а также клетка, из которой мы пришли, и символ хода. Когда из очереди извлекается клетка с минимальной текущей стоимостью, её расстояние уже является наименьшим возможным. Если через неё можно дешевле попасть в соседнюю клетку, стоимость этой клетки обновляется, и она снова добавляется в очередь. Так, переходя от конечной клетки к начальной по сохранённым родителям, можно восстановить один из кратчайших маршрутов.

Алгоритмическая сложность:

По времени: в графе NM вершин и не более $4NM$ рёбер. Алгоритм Дейкстры с очередью с приоритетом работает за $O(NM \log(NM))$. Временная сложность равна $O(NM \log(NM))$.

По памяти: хранятся карта, массив состояний и очередь с приоритетом. Пространственная сложность равна $O(NM)$.

Код:

```
#include <algorithm>
#include <iostream>
#include <queue>
#include <string>
#include <vector>

using namespace std;

const int INF = 1e9;

struct NodeState {
    int cost = INF;
    int parent_r = -1;
    int parent_c = -1;
    char step_char = ' ';
};

struct Position {
    int r, c, dist;

    bool operator>(const Position& other) const {
        return dist > other.dist;
    }
};
```

```

int main() {
    int n, m, sx, sy, tx, ty;
    cin >> n >> m >> sx >> sy >> tx >> ty;
    --sx;
    --sy;
    --tx;
    --ty;

    vector<string> grid(n);
    for (int i = 0; i < n; ++i) {
        cin >> grid[i];
    }

    vector<vector<NodeState>> state(n, vector<NodeState>(m));
    priority_queue<Position, vector<Position>, greater<Position>> pq;

    int dr[4] = {-1, 1, 0, 0};
    int dc[4] = {0, 0, -1, 1};
    char dir[4] = {'N', 'S', 'W', 'E'};

    state[sx][sy].cost = 0;
    pq.push({sx, sy, 0});

    while (!pq.empty()) {
        Position cur = pq.top();
        pq.pop();

        if (cur.dist != state[cur.r][cur.c].cost) {
            continue;
        }

        if (cur.r == tx && cur.c == ty) {
            break;
        }

        for (int k = 0; k < 4; ++k) {
            int nr = cur.r + dr[k];
            int nc = cur.c + dc[k];

            if (nr < 0 || nr >= n || nc < 0 || nc >= m || grid[nr][nc] == '#') {
                continue;
            }

            int w = (grid[nr][nc] == '.') ? 1 : 2;
            int nd = cur.dist + w;

            if (nd < state[nr][nc].cost) {
                state[nr][nc].cost = nd;
                state[nr][nc].parent_r = cur.r;
                state[nr][nc].parent_c = cur.c;
            }
        }
    }
}

```

```

        state[nr][nc].step_char = dir[k];
        pq.push({nr, nc, nd});
    }
}

if (state[tx][ty].cost == INF) {
    cout << -1 << '\n';
    return 0;
}

cout << state[tx][ty].cost << '\n';

string path;
int r = tx, c = ty;
while (r != sx || c != sy) {
    path.push_back(state[r][c].step_char);
    int pr = state[r][c].parent_r;
    int pc = state[r][c].parent_c;
    r = pr;
    c = pc;
}

reverse(path.begin(), path.end());
cout << path << '\n';

return 0;
}

```

Задача №14 «N. Свинки-копилки»

Описание:

Даны N свинок-копилков. Для каждой копилки известно, в какой другой копилке лежит её ключ. Нужно определить минимальное количество копилков, которые необходимо разбить, чтобы затем открыть все остальные.

Доказательство:

Рассмотрим граф, где из вершины i ведёт ребро в копилку, в которой лежит ключ от i . У каждой вершины ровно одно исходящее ребро, поэтому каждая компонента такого графа содержит ровно один цикл. Пока цикл не разорван, ни одну копилку из него нельзя открыть ключом, значит в каждом цикле нужно разбить хотя бы одну копилку.

В DSU последовательно объединяются вершины i и $key_location$. Если они уже находятся в одном множестве, значит добавляемое ребро замыкает цикл, и счётчик циклов увеличивается. Так как в каждой компоненте ровно один цикл, итоговое число таких случаев и равно минимальному количеству копилков, которые нужно разбить.

Алгоритмическая сложность:

По времени: для каждой из N копилков выполняется операции по поиску и объединению, которые составляет $O(\log N)$. Временная сложность равна $O(N \log N)$.

По памяти: хранится массив родителей размера N , поэтому пространственная сложность равна $O(N)$.

Код:

```
#include <iostream>
#include <numeric>
#include <vector>

using namespace std;

struct DisjointSetUnion {
    vector<int> parent;
    int cycles;

    DisjointSetUnion(int n) {
        parent.resize(n + 1);
        for (int i = 0; i <= n; ++i) {
            parent[i] = i;
        }
        cycles = 0;
    }

    int find_set(int i) {
        if (parent[i] == i) {
            return i;
        }
        parent[i] = find_set(parent[i]);
        return parent[i];
    }

    void unite_sets(int i, int j) {
        int root_i = find_set(i);
        int root_j = find_set(j);

        if (root_i != root_j) {
            parent[root_i] = root_j;
        } else {
            cycles++;
        }
    }
};

int main() {
    int n;
    if (!(cin >> n)) {
        return 0;
    }

    DisjointSetUnion dsu(n);
```

```

for (int i = 1; i <= n; ++i) {
    int key_location;
    if (cin >> key_location) {
        dsu.unite_sets(i, key_location);
    }
}

cout << dsu.cycles << endl;

return 0;
}

```

Задача №15 «О. Долой списывание!»

Описание:

Даны N лкшат и M пар лкшат, которые обменивались записками. Нужно определить, можно ли разделить всех лкшат на две группы так, чтобы в каждой паре обмен происходил только между лкшатами из разных групп.

Доказательство:

Построим неориентированный граф, где вершины - лкшата, а рёбра - пары, обменивавшиеся записками. Тогда задача сводится к проверке, является ли граф двудольным. Для этого граф раскрашивается в два цвета с помощью обхода в ширину: если вершина ещё не окрашена, ей назначается цвет, а всем её соседям - противоположный.

Если в процессе обхода найдётся ребро, соединяющее две вершины одного цвета, то разделить лкшат на две группы невозможно. Если весь граф удалось раскрасить без противоречий, то такое разбиение существует.

Алгоритмическая сложность:

По времени: каждая вершина и каждое ребро просматриваются не более одного раза, поэтому временная сложность равна $O(N + M)$.

По памяти: хранятся список смежности, массив цветов и очередь для BFS, поэтому пространственная сложность равна $O(N + M)$.

Код:

```

#include <iostream>
#include <queue>
#include <vector>

using namespace std;

int main() {
    int n;
    int m;
    if (!(cin >> n >> m)) {
        return 0;
    }
}

```

```

vector<vector<int>> g(n + 1);
for (int i = 0; i < m; ++i) {
    int u;
    int v;
    if (cin >> u >> v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
}

vector<int> color(n + 1, -1);

for (int start = 1; start <= n; ++start) {
    if (color[start] == -1) {
        queue<int> q;
        color[start] = 0;
        q.push(start);

        while (!q.empty()) {
            int v = q.front();
            q.pop();

            for (int to : g[v]) {
                if (color[to] == -1) {
                    color[to] = 1 - color[v];
                    q.push(to);
                } else {
                    if (color[to] == color[v]) {
                        cout << "NO" << endl;
                        return 0;
                    }
                }
            }
        }
    }
}

cout << "YES" << endl;
return 0;
}

```

Задача №16 «Р. Авиаперелёты»

Описание:

Дана матрица расхода топлива между каждой парой городов. Нужно найти минимальный размер бака, при котором самолёт сможет долететь из любого города в любой другой, возможно, с промежуточными посадками.

Доказательство:

Зафиксируем некоторый размер бака X . Тогда можно оставить только те перелёты, для которых расход топлива не превышает X . После этого задача сводится к проверке, можно ли по таким перелётам добраться из любого города в любой другой, то есть является ли полученный ориентированный граф сильно связным.

В программе это проверяется двумя обходами из одной вершины: один по исходному графу, второй по обратному. Если в обоих случаях достижимы все вершины, то граф сильно связан. Если при баке X это возможно, то при любом большем баке тоже возможно, потому что допустимых перелётов станет только больше. Значит, ответ обладает свойством монотонности, поэтому по нему можно выполнить бинарный поиск.

Алгоритмическая сложность:

По времени: одна проверка делает два обхода графа по матрице смежности, то есть работает за $O(N^2)$. Бинарный поиск по ответу выполняется за $O(\log C)$, где C - максимальный возможный расход топлива, таким образом временная сложность равна $O(N^2 \log C)$.

По памяти: хранится матрица расходов размера $N \times N$ и вспомогательные массивы для обхода. Пространственная сложность равна $O(N^2)$.

Код:

```
#include <algorithm>
#include <fstream>
#include <iostream>
#include <vector>

using namespace std;

bool can_reach_all(int capacity, const vector<vector<int>>& fuel, bool reverse_mode) {
    int n = static_cast<int>(fuel.size());
    if (n <= 1) {
        return true;
    }

    vector<int> q(n);
    vector<bool> visited(n, false);

    int head = 0;
    int tail = 0;
    q[tail++] = 0;
    visited[0] = true;

    while (head < tail) {
        int current = q[head++];

        for (int neighbor = 0; neighbor < n; ++neighbor) {
            if (visited[neighbor]) {
                continue;
            }

            int cost = reverse_mode ? fuel[neighbor][current] : fuel[current][neighbor];
```

```

    if (cost <= capacity) {
        visited[neighbor] = true;
        q[tail++] = neighbor;
    }
}

return tail == n;
}

int main() {
    ifstream input_file("avia.in");
    ofstream output_file("avia.out");

    int city_count;
    if (!(input_file >> city_count)) {
        return 0;
    }

    if (city_count <= 1) {
        output_file << 0 << '\n';
        return 0;
    }

    vector<vector<int>> fuel(city_count, vector<int>(city_count));
    for (int i = 0; i < city_count; ++i) {
        for (int j = 0; j < city_count; ++j) {
            input_file >> fuel[i][j];
        }
    }

    int left = 0;
    int right = 1000000000;
    int answer = right;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (can_reach_all(mid, fuel, false) && can_reach_all(mid, fuel, true)) {
            answer = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }

    output_file << answer << '\n';
    return 0;
}

```